Module - 13 & 14:
Containerization & K8s Core

- Amjad Hossain
17th May, 2026

The World Before Containerization

Before containers, applications were tightly coupled with -

- Servers
- Operating Systems
- Runtime versions
- manually installed dependencies
- environment-specific configuration
- Logging / Log files

But… but… it works on my machine!!

Developer Laptop/Desktop on one side

production server on the other side

both with different -

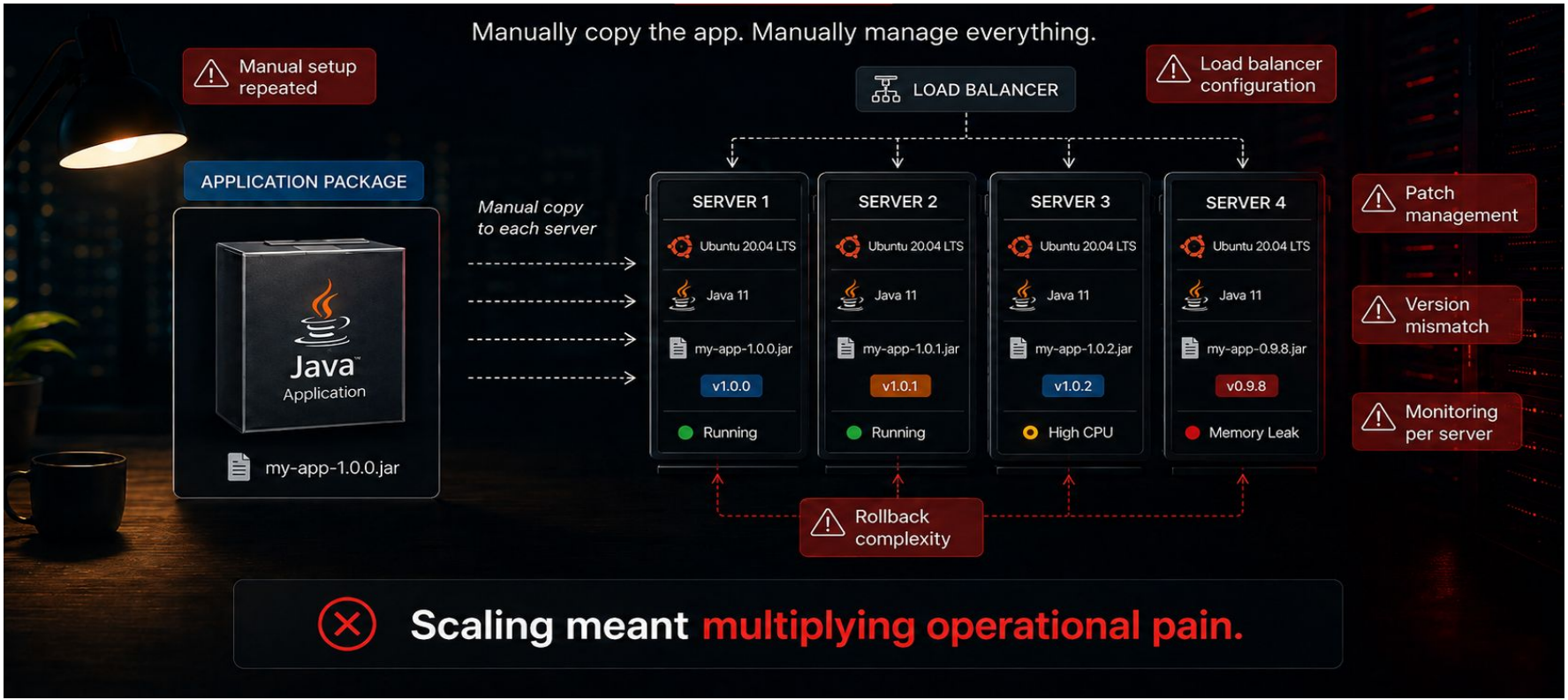
- OS
- Runtime
- Configurations

Common Problems Before Containers

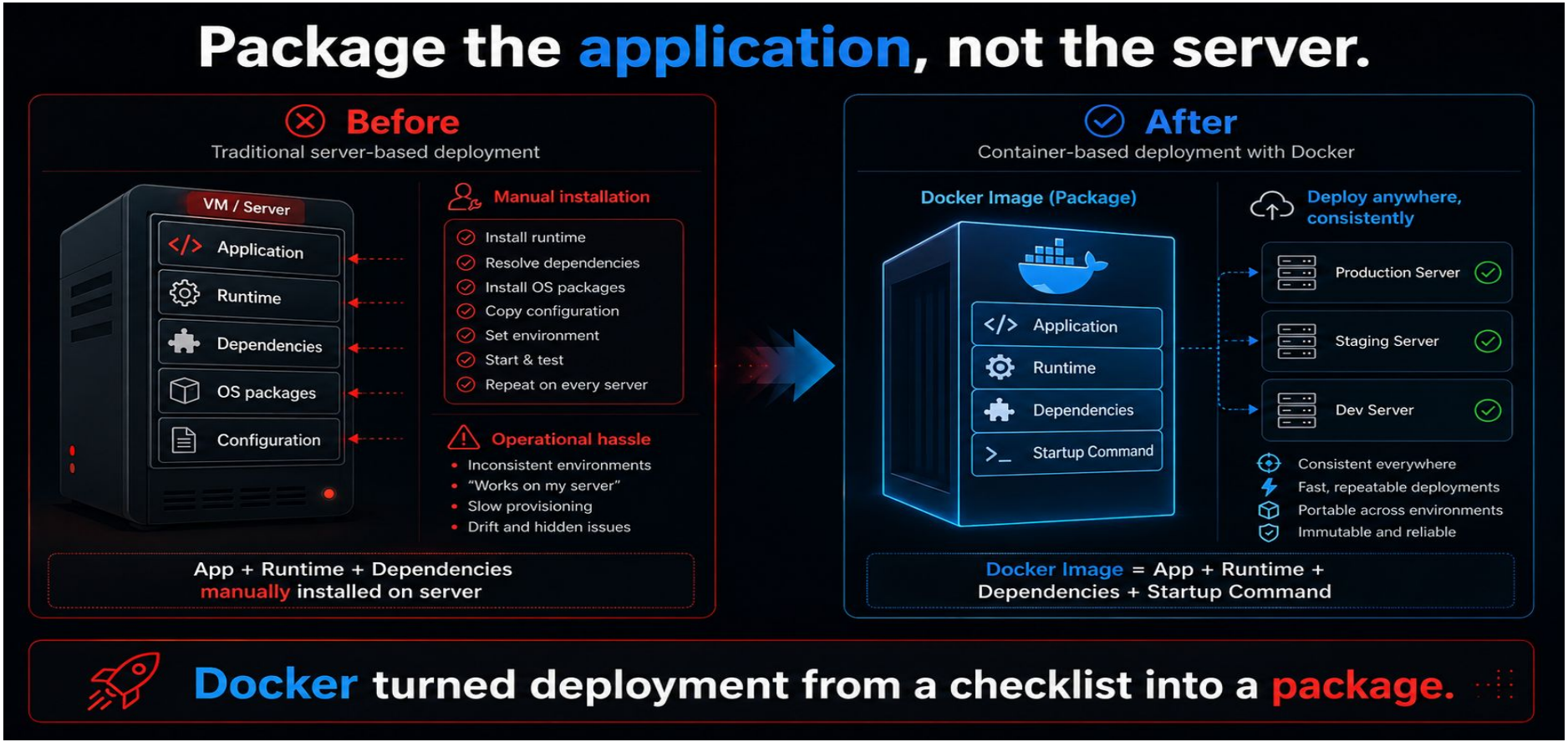
- Different Java/.NET/Node versions
- Missing OS dependencies
- Manual installation steps
- Environment drift
- Hard rollback
- Inconsistent Local/Dev, Staging and Production

Scaling Before Containers

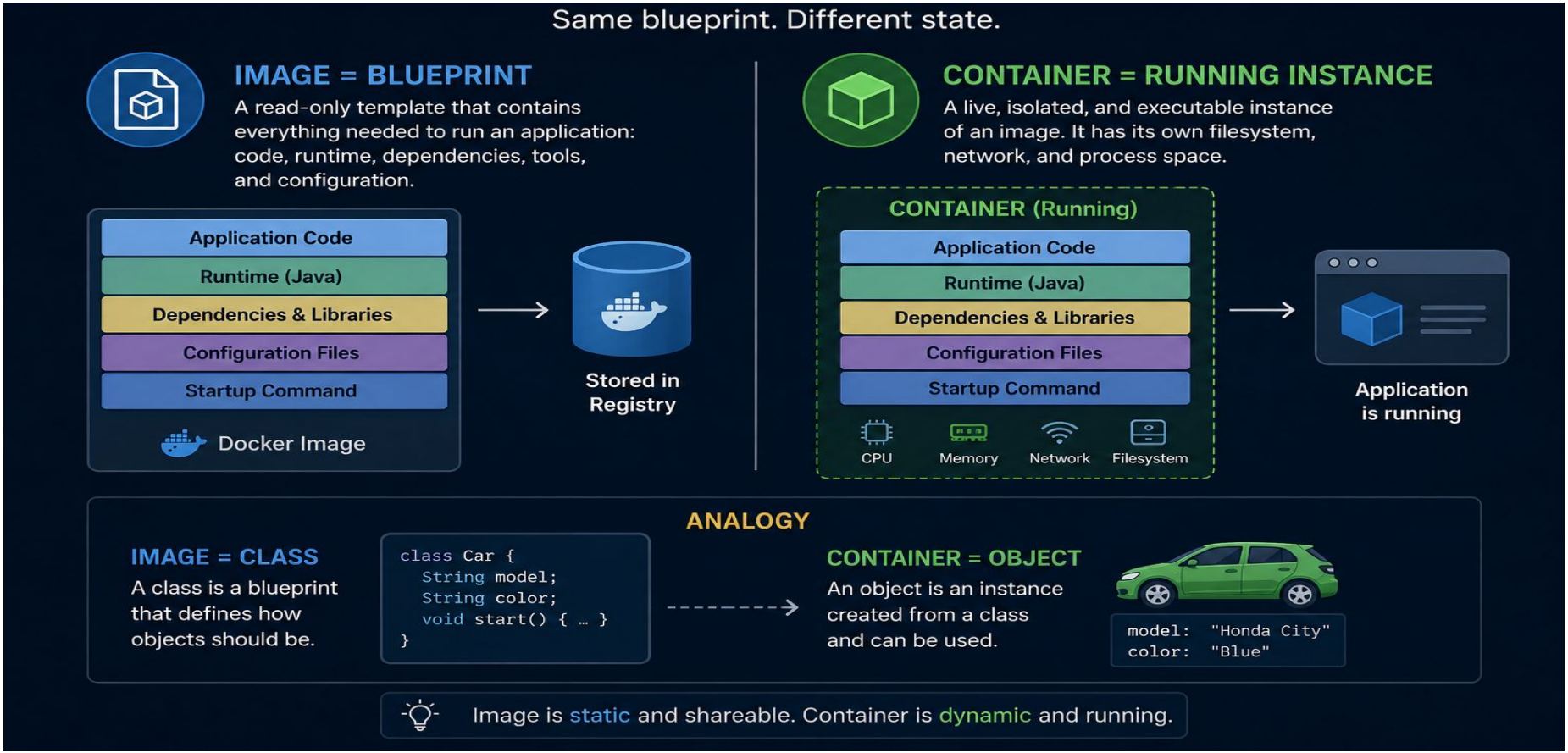
One app copied manually to multiple servers



What Containerization Changed

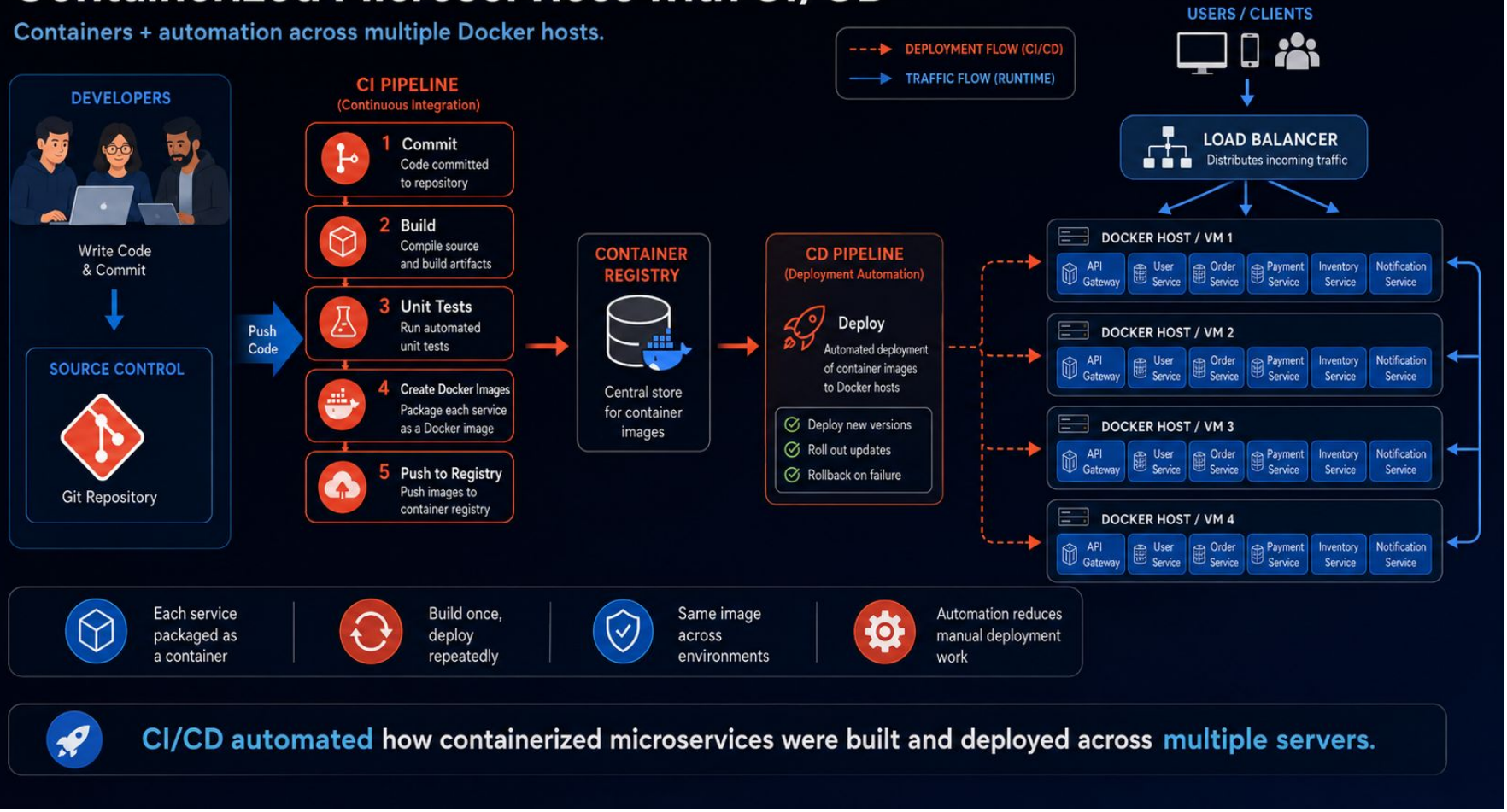


Docker Image vs Container



Containerized Microservices with CI/CD

Containers + automation across multiple Docker hosts.



Each service packaged as a container

Build once, deploy repeatedly

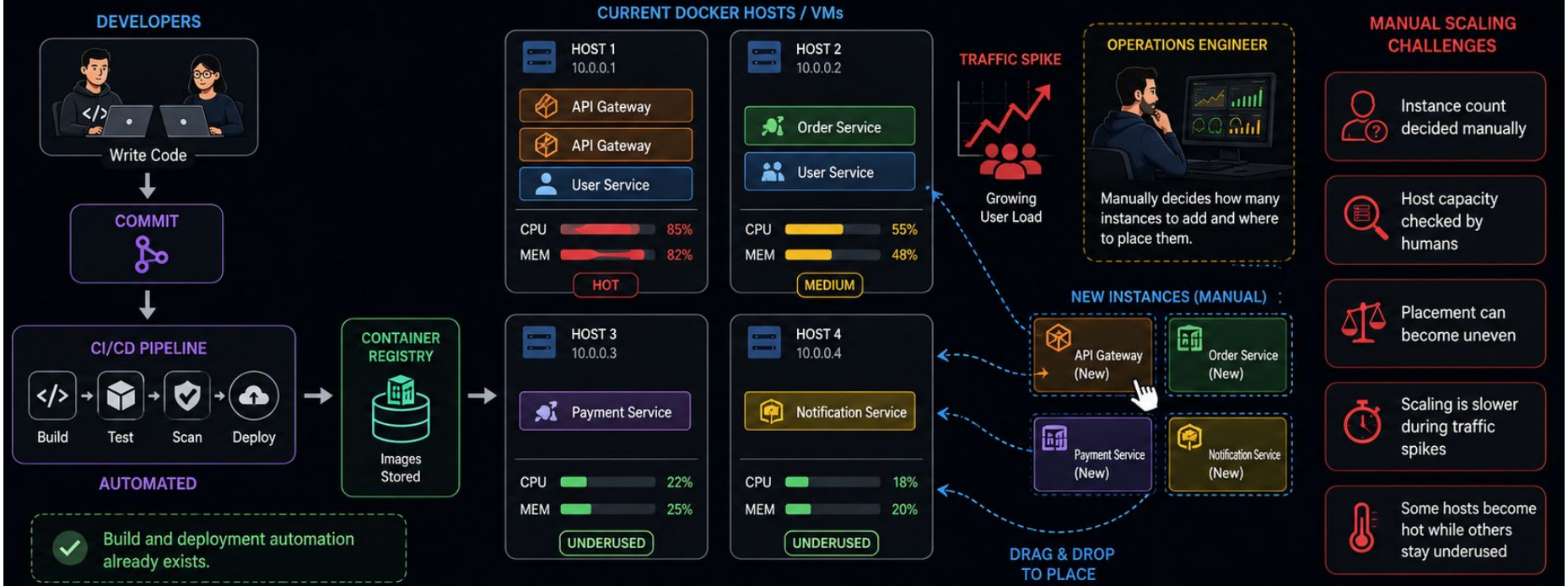
Same image across environments

Automation reduces manual deployment work

CI/CD automated how containerized microservices were built and deployed across **multiple servers.**

Manual Scaling and Placement Challenges

CI/CD can build and deploy containers, but deciding how many instances to run and where to place them is still manual.



Automation built the containers. Operations still had to scale and place them.

Service Discovery and Traffic Routing Challenges

CI/CD can deploy containers, but keeping traffic routes and service endpoints synchronized across many hosts is still operational work.

EXAMPLE: SERVICE INSTANCE REPLACED

OLD INSTANCE (DOWN)

Order Service
10.0.1.15:8080

NEW INSTANCE (UP)

Order Service
10.0.3.27:8080

Endpoint changed.
Routing tables must be updated
to send traffic to the new instance.

Requests

LOAD BALANCER / REVERSE PROXY

- Terminates connections
- Applies route rules
- Health-aware routing

Routes only to healthy instances
- ✓ Performs health checks
- ✓ Avoids unhealthy endpoints
- ✓ Prevents failed requests

CONTAINERS CAN

- Restart
- Move to another host
- Be replaced

IP/Port endpoints change
Routing and discovery
must keep up.

DOCKER HOST / VM 1
10.0.0.1

API Gateway 10.0.0.1:8080	User Service 10.0.0.1:8081
Order Service 10.0.0.1:8082	Inventory Service 10.0.0.1:8083
Payment Service 10.0.0.1:8084	Notification Service 10.0.0.1:8085

HEALTH: ALL HEALTHY

DOCKER HOST / VM 2
10.0.0.2

API Gateway 10.0.0.2:8080	User Service 10.0.0.2:8081
Order Service 10.0.0.2:8082 (STOPPED)	Inventory Service 10.0.0.2:8083
Payment Service 10.0.0.2:8084	Notification Service 10.0.0.3:8085

HEALTH: DEGRADED

DOCKER HOST / VM 3
10.0.0.3

API Gateway 10.0.0.3:8080	User Service 10.0.0.3:8081
Order Service NEW 10.0.3.27:8080	Inventory Service 10.0.0.3:8083
Payment Service 10.0.0.3:8084	Notification Service 10.0.0.3:8085

HEALTH: ALL HEALTHY

DOCKER HOST / VM 4
10.0.0.4

API Gateway 10.0.0.4:8080	User Service 10.0.0.4:8081
Order Service 10.0.0.4:8082	Inventory Service 10.0.0.4:8083
Payment Service 10.0.0.4:8084	Notification Service 10.0.0.4:8085

HEALTH: ALL HEALTHY

EAST-WEST SERVICE-TO-SERVICE COMMUNICATION

Services call each other across hosts. Paths change as instances move or restart.

- North-South Traffic (Client → Services)
- East-West Traffic (Service → Service)
- Health Checks
- Stale / Failed Route

KEY CHALLENGES

Dynamic service endpoints
IPs and ports change as instances move or restart.

Service registration & deregistration
Endpoints must be added and removed in real time.

Health-aware routing
Traffic must go only to healthy instances.

Keeping route rules up to date
Routing tables must reflect the current state.

East-west communication
Services talk across hosts; paths are dynamic.

Harder troubleshooting
Changing paths and endpoints make issues harder to trace.

OPERATIONS / DEVOPS ENGINEER

ROUTING TABLES (AT LOAD BALANCER)

/api/*	API Gateway	Host 1,2,3,4
/users/*	User Service	Host 1,2,3,4
/orders/*	Order Service	Host 1,3,4
/payments/*	Payment Service	Host 1,2,3,4

SERVICE ENDPOINTS (DISCOVERED)

Service	Healthy Instances	Endpoints
API Gateway	4	10.0.0.1:8080, 10.0.0.2:8080, 10.0.0.3:8080, 10.0.0.4:8080
User Service	4	10.0.0.1:8081, 10.0.0.2:8081, 10.0.0.3:8081, 10.0.0.4:8081
Order Service	3	10.0.0.1:8082, 10.0.3.27:8080, 10.0.0.4:8082
Payment Service	4	10.0.0.1:8084, 10.0.0.2:8084, 10.0.0.3:8084, 10.0.0.4:8084

HEALTH STATUS

Healthy	Routing allowed
Degraded	Limited routing
Unhealthy	No routing
Unknown	Probing...

STALE ROUTE EXAMPLE

Request to Order Service
via Host 2 (10.0.0.2:8082)

TIMEOUT / REQUEST FAILED

Instance is down.
Route is stale.

As containers changed location and state, routing and discovery had to stay in sync.

Rollout, Update, and Rollback Coordination Challenges

CI/CD automates delivery, but coordinating safe multi-host, multi-service releases is still complex.

1. BUILD NEW VERSION

Developers

CI/CD PIPELINE

Build → Test → Scan → Push

2. PUSH IMAGE

Container Registry

myapp:v2.3

3. DEPLOYMENT WAVES

Automated deployment, strategy and coordination are the challenge.

MICROSERVICES

- API Gateway
- Order Service
- Payment Service
- Notification Service
- User Service

VERSION LEGEND

- v2.3 New Version
- v2.2 Previous Version
- ✕ Failed / Unhealthy

WAVE 1 — HOST 1 — 10.0.0.1

UPDATED SUCCESSFULLY

API Gateway	v2.3
Order Service	v2.3
Payment Service	v2.3
Notification Service	v2.3
User Service	v2.3

Health: Healthy
Traffic: 25%

WAVE 2 (IN PROGRESS) — HOST 2 — 10.0.0.2

MID-ROLLOUT

API Gateway	v2.3
Order Service	v2.3
Payment Service	v2.2
Notification Service	v2.2
User Service	v2.2

Health: Degraded (1 check failing)
Traffic: 25% (steady)

WAVE 3 (FAILED) — HOST 3 — 10.0.0.3

ROLLOUT FAILED

API Gateway	v2.3
Order Service	v2.2
	✕
Notification Service	v2.2
User Service	v2.2

Health: Unhealthy
Traffic: 0% (drained)

WAVE 4 (PENDING) — HOST 4 — 10.0.0.4

PENDING

API Gateway	v2.2
Order Service	v2.2
Payment Service	v2.2
Notification Service	v2.2
User Service	v2.2

Health: Healthy
Traffic: 25%

TRAFFIC DISTRIBUTION (EXAMPLE)

25% | 25% | 0% | 25%

WHY ROLLOUT IS STILL CHALLENGING

- Rolling out across many hosts in the correct sequence
- Related services may need coordinated upgrade order
- Partial rollout failure can leave mixed versions running
- Health checks and validation are needed before continuing
- Rolling back means reverting changes across multiple hosts/services, not just clicking deploy
- Avoiding downtime and traffic disruption during release

OPERATIONS / RELEASE ENGINEER

MONITORING & VALIDATION

- Monitor service health, logs, and metrics
- Run synthetic and real user checks
- Validate dependencies and data flows
- Decide: Continue, Pause, or Rollback

ROLLBACK COORDINATION (IF NEEDED)

Identify failing services/hosts → Drain traffic from affected hosts → Revert services to previous version (v2.2) → Re-run health checks and validate → Restore traffic gradually and monitor

Repeat across impacted hosts and dependent services

KEY TAKEAWAY

CI/CD handles build, test, and deploy. Runtime coordination—order, health, validation, rollback, and traffic management—still requires careful planning and human judgment.

CI/CD automated delivery. Safe rollout and rollback still required careful coordination.

Failure Recovery and Self-Healing Gaps

A container can restart on one host, but recovering from host failure or restoring missing capacity across hosts still needs coordination.

USERS

LOAD BALANCER

TRAFFIC REDISTRIBUTED

Before Failure → After Failure

Traffic shifts to healthy hosts, increasing load.

HOST 1 / VM
- API Service
- User Service
- Order Service
- Payment Service

CPU 78%
MEM 76%
HIGH LOAD

HOST 2 / VM
- API Service
- User Service
- Order Service
- Payment Service

CPU 82%
MEM 79%
HIGH LOAD

HOST 3 / VM (FAILED)
- API Service UNAVAILABLE
- User Service UNAVAILABLE
- Order Service UNAVAILABLE
- Payment Service UNAVAILABLE

CPU 0%
MEM 0%
HOST DOWN

HOST 4 / VM
- API Service
- User Service
- Order Service
- Payment Service

CPU 80%
MEM 77%
HIGH LOAD

Basic restart policies can restart containers on the same host if they crash. They cannot recover from host failure. Cluster-wide recovery is still manual.

OPERATIONS ENGINEER

ALERT RECEIVED
- Host 3 is down
- Services unavailable
- Traffic redistributed
- Load is high on Hosts 1, 2, 4

MANUAL DECISION & ACTIONS
- Review alerts and impact
- Check capacity on other hosts
- Choose target host(s)
- Redeploy missing services
- Verify traffic & service health

SELF-HEALING GAPS
- Single container restart on the same host is not enough when the entire host fails
- Failed workloads must be restarted or redeployed elsewhere
- Recovery needs spare capacity on other hosts
- Traffic must be rebalanced after failure
- Teams must verify service health after recovery
- Recovery time is slower when placement and restart decisions are manual

1 DETECT HOST FAILURE
- Monitoring detects Host 3 down
- Services marked unavailable and alerts triggered

2 CHOOSE REPLACEMENT HOST
- Engineer checks capacity
- Selects Hosts 1, 2, and 4 to take over workloads

3 REDEPLOY MISSING SERVICES
- Restart or redeploy containers from Host 3
- Requires spare capacity

4 RESTORE TRAFFIC
- Load balancer updates routing
- Traffic rebalanced across healthy hosts

5 VERIFY SERVICE HEALTH
- Validate service health and performance
- Confirm recovery is complete

Container restart is not the same as cluster-wide self-healing.

Configuration, Secrets, and Environment Coordination Challenges

Central config and secret stores help, but coordinating versions, rollout timing, and environment-specific settings across many services is still hard.

LEGEND
- Config (v18)
- Secrets
- Env-Specific Values

CONFIGURATION SERVICE
Active Version: v18
Change History: v15 → v16 → v17 → v18

SECRET MANAGER / VAULT
Secret Set: payments-db
Status: Rotating (v7 → v8)

DEV	TEST	STAGING	PROD
Docker Hosts / VMs	Docker Hosts / VMs	Docker Hosts / VMs	Docker Hosts / VMs
API Service: Config: v18, Secret: v7, RELOAD OK	API Service: Config: v18, Secret: v7, RELOAD OK	API Service: Config: v18, Secret: v8, RELOADED	API Service: Config: v18, Secret: v8, RELOADED
User Service: Config: v18, Secret: v7, RELOAD OK	Billing Service: Config: v18, Secret: v7, RELOAD OK	Payment Service: Config: v17, Secret: v7, PENDING RESTART	Billing Service: Config: v18, Secret: v8, RELOADED
Worker Service: Config: v18, Secret: v7, RELOAD OK	Worker Service: Config: v18, Secret: v7, RELOAD OK	Worker Service: Config: v18, Secret: v8, RELOADED	Worker Service: Config: v18, Secret: v8, RELOADED
</> Env-Specific Values (dev.*)	</> Env-Specific Values (test.*)	</> Env-Specific Values (staging.*)	</> Env-Specific Values (prod.*)

KEY CHALLENGES
- Centralized config exists, but many services must consume the correct version.
- Secret rotation must propagate safely across running services.
- Environment-specific values must remain aligned across environments.
- Some config changes require controlled restarts or reloads.
- Teams must know which version of config is currently active where.
- A bad config rollout can affect many services quickly.

OPERATIONS / DEVOPS

- Monitor config versions and secret status
- Track rollout progress across environments
- Validate reloads / restarts
- Respond to issues quickly

CONFIG & SECRET STATUS OVERVIEW

ENVIRONMENT	CONFIG VERSION (ACTIVE)				SECRET SET: payments-db			LAST UPDATED
	API SERVICE	BILLING SERVICE	PAYMENT SERVICE	WORKER SERVICE	CURRENT	NEXT	ROTATION STATUS	
DEV	v18	v18	v18	v18	v7	v8	In Progress	2m ago
TEST	v18	v18	v18	v18	v7	v8	In Progress	3m ago
STAGING	v18	v18	v17	v18	v7	v8	In Progress (1 service pending)	5m ago
PROD	v18	v18	v18	v18	v8	v9	Not Started	1m ago

RECENT CHANGES

10:21	Deployed config v18 to PROD
10:19	Secret rotation v8 started (payments-db)
10:17	Deployed config v18 to STAGING
10:12	Config change: timeout increased

Centralized config helps. Coordinating versions and secrets across many services is still operational work.

Observability, Capacity, and Resource Optimization Challenges

Monitoring tools provide visibility, but teams still have to correlate signals, manage host hot spots, and decide where containers should run.

TELEMETRY SOURCES

- Microservices Containers
- Hosts / VMs
- Network & Ingress
- Datastores & Queues
- External Services

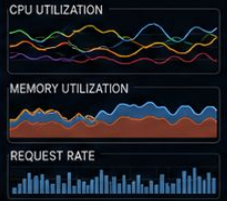
CENTRALIZED OBSERVABILITY STACK / DASHBOARDS

LOGS

10:01:23	ERROR	Payment Service
10:01:23	ERROR	Order Service
10:01:24	WARN	Inventory Service
10:01:24	ERROR	Payment Service
10:01:25	INFO	User Service

... many thousands more

METRICS



TRACES

Duration: 2.45s (High)

API Gateway	120ms
Order Service	380ms
Payment Service	980ms
Inventory Service	450ms
Notification Service	210ms

ALERTS

- High CPU on Host 1
- Payment Service High Latency
- Pod Restarts (Order Service)
- High Memory on Host 1

... 23 more alerts

Dashboards · Alerting · Correlation · Reports

OPERATIONS / DEVOPS ENGINEER

TRYING TO FIND THE ROOT CAUSE

- Which service is causing high CPU?
- Is it traffic, code, or a dependency?
- Should we move workloads?
- Do we have spare capacity?
- What will be the impact?

DOCKER HOSTS / VMs

HOST 1 — 10.0.0.1 — OVERLOADED	HOST 2 — 10.0.0.2 — BALANCED	HOST 3 — 10.0.0.3 — UNDERUSED	HOST 4 — 10.0.0.4 — BALANCED
API Gateway	User Service	Reporting Service	Auth Service
Payment Service	Catalog Service	Analytics Service	Profile Service
Order Service	Cart Service	Batch Service	Settings Service
Inventory Service	Search Service	Export Service	Email Service
Notification Service	Recommendation Service	Admin Service	Audit Service
...	...	...	...
CPU 95%	CPU 58%	CPU 18%	CPU 52%
MEM 93%	MEM 54%	MEM 22%	MEM 49%
Hot Spot: High CPU & Memory	Healthy & Balanced	Underutilized Capacity	Healthy & Balanced

> 80% (High) · 50% - 80% (Medium) · < 50% (Low)

MANY SIGNALS TO CORRELATE

- Metrics, logs, traces, alerts
- Per-service and per-host views
- SLOs, error budgets, SLIs
- Infrastructure and workloads
- External dependencies

High cardinality = noise and complexity

KEY CHALLENGES

- Centralized monitoring exists, but there are still many signals to correlate
- Root-cause analysis across hosts and services can take time
- Some hosts become hot while others stay underused
- Capacity planning and placement decisions remain manual
- Noisy alerts and many dashboards can slow response
- Teams need to understand both service health and host utilization

Visibility helps. Efficient placement, troubleshooting, and optimization still take constant coordination.

Introducing Kubernetes

The Orchestrator for Containerized Applications

Kubernetes automates deployment, scaling, and operations of containerized applications across a cluster of machines.

What You Run Today

- **Containers** — Your application packaged
- **CI/CD Pipeline** — Build, test and deploy containers
- **Storage** — Data that persists
- **Networking** — Connect your services
- **Configs & Secrets** — App settings and sensitive data

Kubernetes Orchestrates Everything

- Automated Scheduling
- Self-Healing
- Auto Scaling
- Rolling Updates & Rollbacks
- Service Discovery & Load Balancing
- Storage Orchestration

Cluster of Machines

Worker Node	Worker Node	...	Worker Node
CPU, Memory, Disk, Network	CPU, Memory, Disk, Network		CPU, Memory, Disk, Network

What You Get

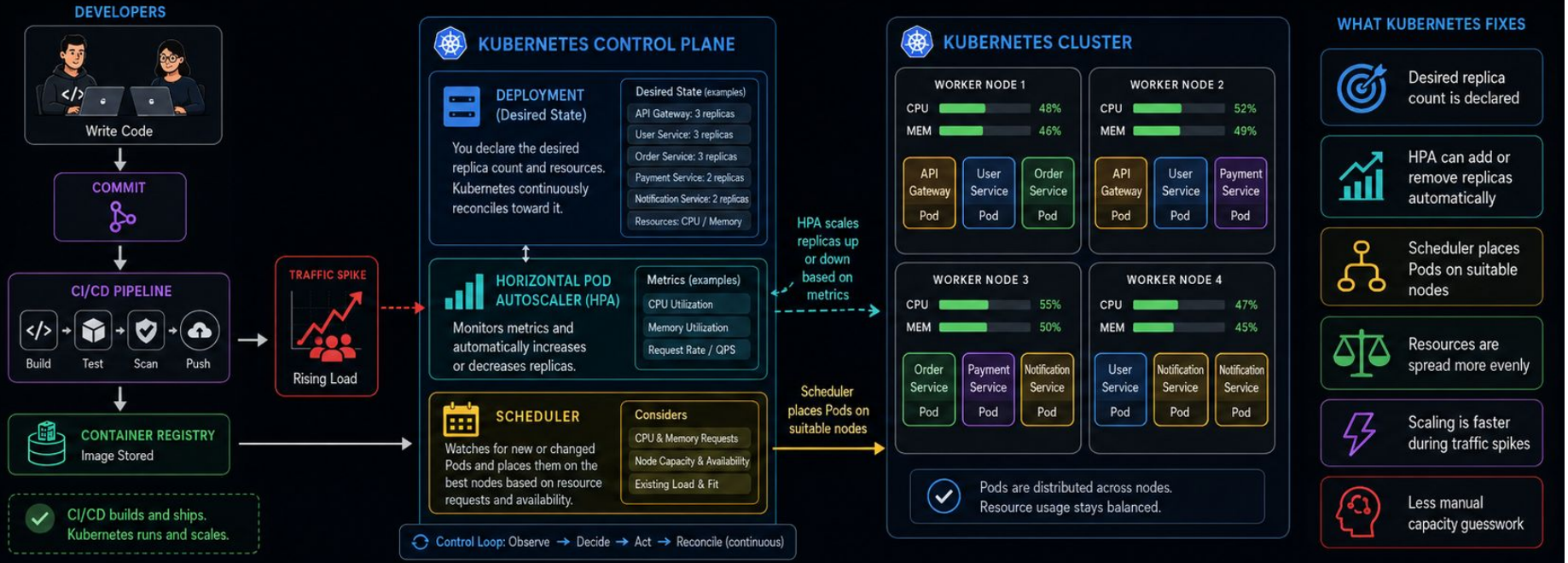
- **Run at Scale** — Seamlessly handle growth and traffic
- **High Availability** — Built-in resilience and self-healing
- **Operational Simplicity** — Automate complex infrastructure tasks
- **Portability** — Run anywhere: cloud, on-prem, hybrid
- **Consistent & Observable** — Unified view, metrics, logs, and health

Kubernetes takes care of the undifferentiated heavy lifting, so you can focus on building and delivering great applications.

Build → Ship → Delight Users

Kubernetes Solves Manual Scaling and Placement

CI/CD still builds and ships the image. Kubernetes decides how many replicas should run and where they should be placed.



WHAT KUBERNETES FIXES

- Desired replica count is declared
- HPA can add or remove replicas automatically
- Scheduler places Pods on suitable nodes
- Resources are spread more evenly
- Scaling is faster during traffic spikes
- Less manual capacity guesswork

Declare once
You define the desired replicas and resource requests in the Deployment.

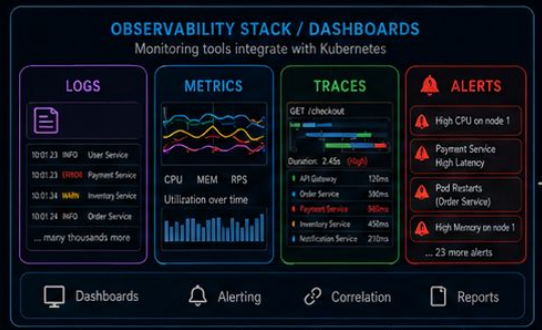
HPA scales automatically
HPA adjusts replica count up or down based on metrics to meet demand.

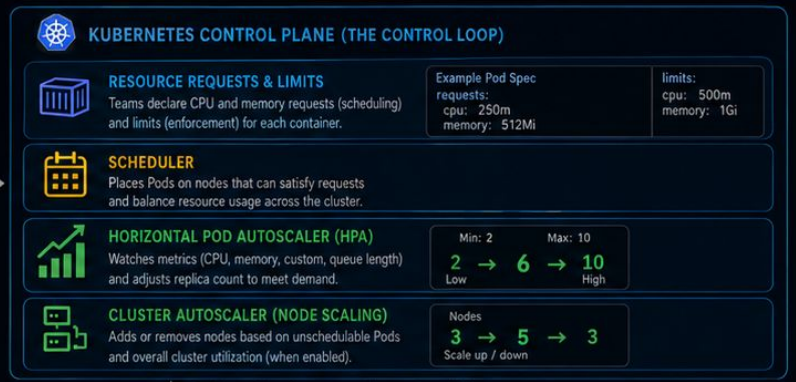
Scheduler places intelligently
Scheduler places Pods on nodes with enough free CPU and memory.

Kubernetes turned scaling and placement from manual operations work into a control loop.

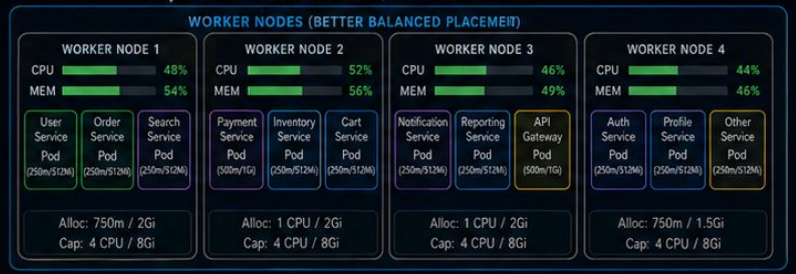
Kubernetes Solves Observability, Capacity, and Resource Optimization

Requests, limits, autoscalers, and standardized health signals make placement and capacity far more systematic.





Decisions drive placement, scaling, and remediation



WHAT KUBERNETES FIXES

Scheduler uses resource requests for better placement
Pods land on nodes with enough requested CPU and memory and are balanced.

Requests and limits improve capacity planning
Provides predictable resource guarantees and prevents noisy neighbor issues.

HPA scales workload replicas from metrics
Automatically scales up and down based on demand to maintain performance.

Cluster-level view standardizes health and status
Pod and node status, events, and conditions are consistent across workloads.

Monitoring tools integrate with Kubernetes labels and events
Standard labels, metrics, and events make correlation and troubleshooting easier.

Hot spots and underused nodes are easier to spot
Dashboards highlight imbalance so teams can act on real signals, not guesswork.

ACCURACY NOTE
Kubernetes is not the entire observability platform. It provides standardized workload, health, and resource signals that make monitoring and capacity management far more effective.

Kubernetes turned placement and scaling into policy-driven control, and gave monitoring tools a consistent cluster view.

Kubernetes Solves Rollout, Update, and Rollback Coordination

CI/CD can trigger the release. Kubernetes performs the rollout safely across replicas and can roll back by revision.

DEVELOPERS

COMMIT

CI/CD PIPELINE

Build → Test → Scan → Push → Deploy Manifest

CONTAINER REGISTRY

Stores images myapp:v2.3

CI/CD builds, tests, scans, pushes the image and deploys the manifest.

KUBERNETES DEPLOYMENT CONTROLLER

Declarative desired state drives safe, gradual updates and rollbacks.

DEPLOYMENT (Revision 12)
Desired Image: myapp:v2.2
Replicas: 4

DESIRED STATE CHANGE

DEPLOYMENT (Revision 13)
Desired Image: myapp:v2.3
Replicas: 4

OLD REPLICASET (rev 12)

Image: myapp:v2.2 Desired: 4

v2.2 Pod Ready | v2.2 Pod Ready | v2.2 Pod Ready | v2.2 Pod Ready

NEW REPLICASET (rev 13)

Image: myapp:v2.3 Desired: 4

v2.3 Pod Starting | v2.3 Pod Not Ready | v2.3 Pod Not Readdy | v2.3 Pod Not Ready

ROLLING UPDATE PROGRESS (Gradual replacement without downtime)

WAVE 1	WAVE 2	WAVE 3	WAVE 4
25%	50%	75%	100%
v2.3 Pod Ready, v2.2 Pod Ready, v2.2 Pod Ready, v2.2 Pod Ready	v2.3 Pod Ready, v2.3 Pod Ready, v2.2 Pod Ready, v2.2 Pod Ready	v2.3 Pod Ready, v2.3 Pod Ready, v2.3 Pod Ready, v2.2 Pod Ready	v2.3 Pod Ready, v2.3 Pod Ready, v2.3 Pod Ready, v2.3 Pod Ready

READINESS PROBES / HEALTH CHECKS

Gate rollout progression

Liveness Probe — Is the app alive? → Readiness Probe — Is the app ready to serve traffic? → Only Ready Pods receive traffic

NO DOWNTIME WITH maxSurge / maxUnavailable

maxSurge: 1
maxUnavailable: 1

Allows a new Pod to come up before an old Pod is removed.

Capacity buffer prevents downtime during updates.

FAILED ROLLOUT SCENARIO

One new Pod fails health checks.

v2.3 Pod Ready | v2.3 Pod Ready | v2.3 Pod Unhealthy | v2.2 Pod Ready

ROLLOUT PAUSES AUTOMATICALLY

- Unhealthy new Pod is removed.
- Progress pauses until healthy Pods are available.
- Traffic continues to go only to Ready Pods.

Trigger Rollback (manual or auto)

ROLLBACK TO PREVIOUS REVISION

Revert desired state to Revision 12 (v2.2).

v2.2 Pod Ready | v2.2 Pod Ready | v2.2 Pod Ready | v2.2 Pod Ready

REVISION HISTORY

Tracked by Deployment

10 v2.0 ✓ → 11 v2.1 ✓ → 12 v2.2 (Currently Active) → 13 v2.3 (Failed) ✗ → Revert to Revision 12

SERVICES CONTINUE TO ROUTE TRAFFIC ONLY TO READY PODS

Service uses Endpoints from Ready Pods.
Healthy traffic during rollout and rollback.

Service → Pod Ready | Pod Ready | Pod Unhealthy

WHAT KUBERNETES ADDS

- Rolling updates handled by Deployment strategy
- Readiness checks control rollout progression
- New and old ReplicaSets are tracked by revision
- Rollout can pause on failure
- Rollback is reverting to a previous ReplicaSet revision
- Less manual coordination across hosts and instances

WHY THIS STILL MATTERS (EVEN WITH CI/CD)

- CI/CD delivers the new image and manifest.
- Kubernetes coordinates runtime rollout order, health validation, traffic safety, and rollback.

CI/CD starts the release. Kubernetes makes the runtime rollout safer, gradual, and reversible.

Kubernetes Solves Configuration and Secrets Coordination

ConfigMaps and Secrets make runtime configuration declarative and repeatable across environments.

CONFIG & SECRET SOURCES

Git / Config Repo

config/
- dev.yaml
- staging.yaml
- prod.yaml

secrets/ (refs)

External Secret Manager

e.g., Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager

Kubernetes pulls / references secrets at runtime.

CONFIGMAP (Config)

api-config

KEY	VALUE (v18)
API_URL	https://api.v18.example.com
FEATURE_X	true
LOG_LEVEL	info
RETRY_COUNT	3
...	...

Versioned Updates

SECRET (DB_CREDENTIALS)

db-credentials

KEY	VALUE (v7)
username	******
password	******
endpoint	db.v18.example.com
...	...

KUBERNETES CLUSTER

API SERVICE	BILLING SERVICE	WORKER SERVICE	PAYMENT SERVICE	WEB UI
Deployment	Deployment	Deployment	Deployment	Deployment
Pods	Pods	Pods	Pods	Pods
Consumes via: • Env Vars (ConfigMap) • Env Vars / Files (Secret)	Consumes via: • Env Vars (ConfigMap) • Env Vars / Files (Secret)	Consumes via: • Env Vars (ConfigMap) • Env Vars / Files (Secret)	Consumes via: • Env Vars (ConfigMap) • Env Vars / Files (Secret)	Consumes via: • Env Vars (ConfigMap) • Env Vars / Files (Secret)

Rolling Update: 1 / 3 pods (waiting for restart)

WHAT KUBERNETES IMPROVES

- Config stays outside the container image.
- Same image can run in multiple environments.
- ConfigMaps and Secrets are referenced declaratively.
- Environment-specific values stay organized by namespace / manifest.
- Config changes can be rolled out in a controlled way.
- Teams can see which version is active where.

ENVIRONMENTS (NAMESPACES)

dev	staging	prod
Active	Active	Pending Rollout
api-config v18	api-config v19	api-config v18
db-credentials v7	db-credentials v8	db-credentials v7
All services running	Rolled out to staging first	Rollout to prod scheduled
Image: myapp:v1.4.0	Image: myapp:v1.4.0	Image: myapp:v1.4.0

VERSION ROLLOUT FLOW

1. Update ConfigMap: v18 → v19
2. Apply to staging namespace
3. Validate & observe
4. Promote to prod namespace
5. Roll out at controlled pace

OPERATORS / DEVOPS

- View active config & secret versions per environment
- See rollout status per service
- Trigger or pause rollouts
- Detect drift or failed rollouts

ACTIVE CONFIG & SECRET VERSIONS AT A GLANCE

ENVIRONMENT	API-CONFIG (ConfigMap)	DB-CREDENTIALS (Secret)	SERVICES					LAST UPDATED
			API	BILLING	WORKER	PAYMENT	WEB UI	
dev	v18	v7	v18	v18	v18	v18	v18	10:12 AM ✓
staging	v19	v8	v19	v19	v19	v19	v19	10:28 AM ✓
prod	v18	v7	v18	v18	v18*	v18	v18	10:08 AM

ROLL OUT STATUS

dev	Up to date	All services ready
staging	Up to date	All services ready
prod	In Progress	1 service rolling out

NOTE

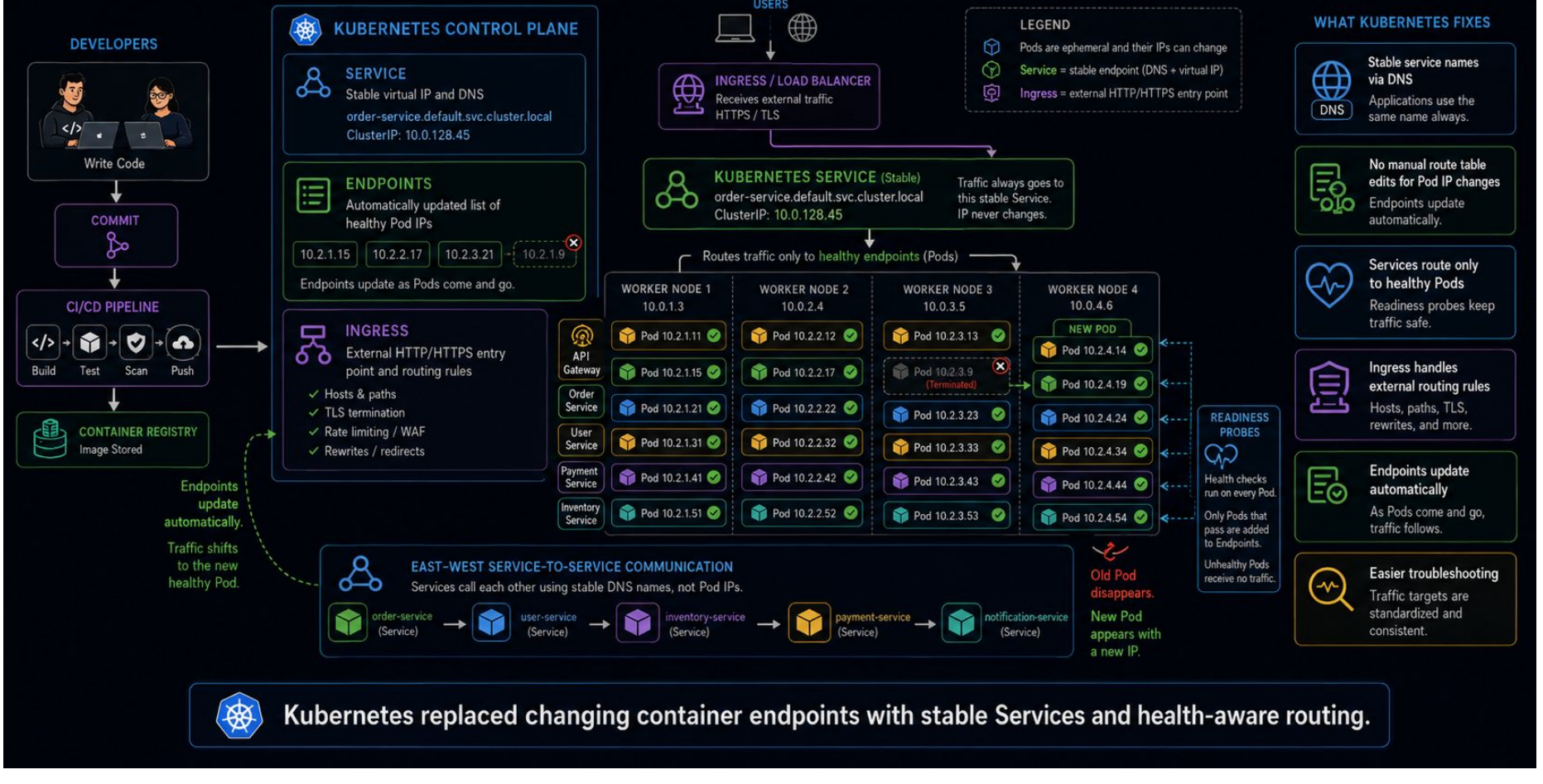
Centralized config and secret stores may already exist.

Kubernetes improves how workloads consume, reference, and roll out that config at runtime.

Kubernetes made configuration and secrets more declarative, repeatable, and easier to coordinate across environments.

Kubernetes Solves Service Discovery and Traffic Routing

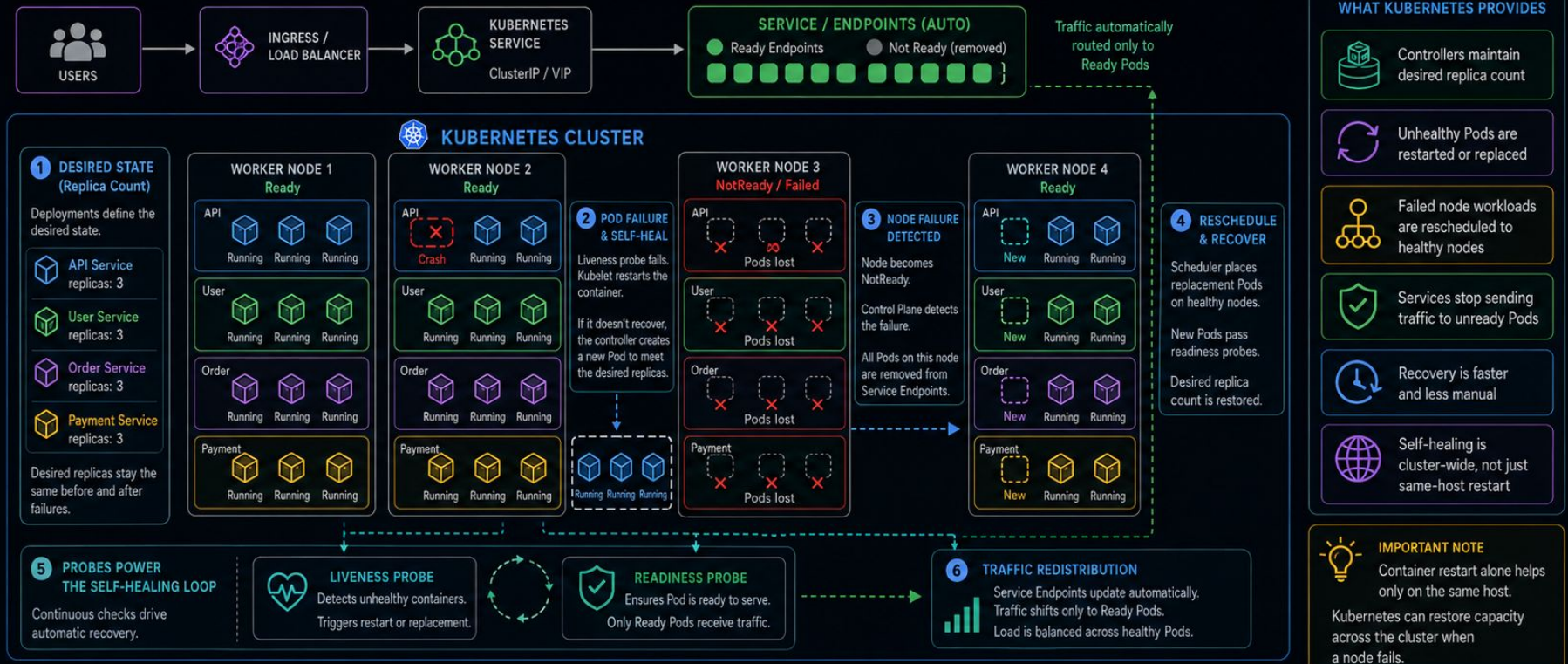
Pods can change IPs. Services and Ingress keep traffic stable and route only to healthy endpoints.



Kubernetes replaced changing container endpoints with stable Services and health-aware routing.

Kubernetes Solves Failure Recovery and Self-Healing

When a Pod or even a node fails, Kubernetes recreates the missing capacity and reroutes traffic.



WHAT KUBERNETES PROVIDES

- Controllers maintain desired replica count
- Unhealthy Pods are restarted or replaced
- Failed node workloads are rescheduled to healthy nodes
- Services stop sending traffic to unready Pods
- Recovery is faster and less manual
- Self-healing is cluster-wide, not just same-host restart

IMPORTANT NOTE

Container restart alone helps only on the same host.

Kubernetes can restore capacity across the cluster when a node fails.

Kubernetes turned failure recovery from manual intervention into a self-healing control loop.

Core Kubernetes Terms

Term	Simple Meaning
Cluster	A group of machines managed together by Kubernetes
Node	A machine inside the cluster where applications run
Worker Node	A node that runs application workloads
Control Plane	The brain of Kubernetes that makes decisions
Pod	The smallest deployable unit in Kubernetes
Container	The actual packaged application running inside a Pod
Deployment	Defines how many copies of an application should run
ReplicaSet	Ensures the required number of Pod replicas are running
Service	Gives Pods a stable network address
Ingress	Handles external HTTP/HTTPS traffic into the cluster

Core Kubernetes Terms… …

Term	Simple Meaning
Namespace	Logical separation inside a cluster
ConfigMap	Stores non-sensitive configuration
Secret	Stores sensitive configuration such as passwords or tokens
Volume	Storage attached to a Pod
PersistentVolume	Cluster-level storage resource
PersistentVolumeClaim	A request for storage by an application
StatefulSet	Manages stateful applications like databases
DaemonSet	Runs one Pod on every node or selected nodes
Job	Runs a task until completion
CronJob	Runs a task on a schedule

Core Kubernetes Terms… …

Term	Simple Meaning
Label	Key-value metadata attached to Kubernetes objects
Selector	Finds objects using labels
Annotation	Extra metadata for tools or controllers
Manifest	YAML file that declares Kubernetes resources
Desired State	What you want the system to look like
Actual State	What is currently running
Reconciliation Loop	Kubernetes continuously tries to make actual state match desired state

Control Plane Terms

Term	Simple Meaning
API Server	Main entry point for all Kubernetes commands
etcd	Database where Kubernetes stores cluster state
Scheduler	Decides which node should run a Pod
Controller Manager	Runs controllers that maintain desired state
Cloud Controller Manager	Integrates Kubernetes with cloud provider resources

Node-Level Terms

Term	Simple Meaning
kubelet	Agent running on each node; talks to the control plane
kube-proxy	Handles networking rules for Services
Container Runtime	Software that runs containers, such as containerd
Image	Packaged application artifact pulled by the runtime

Operational Terms

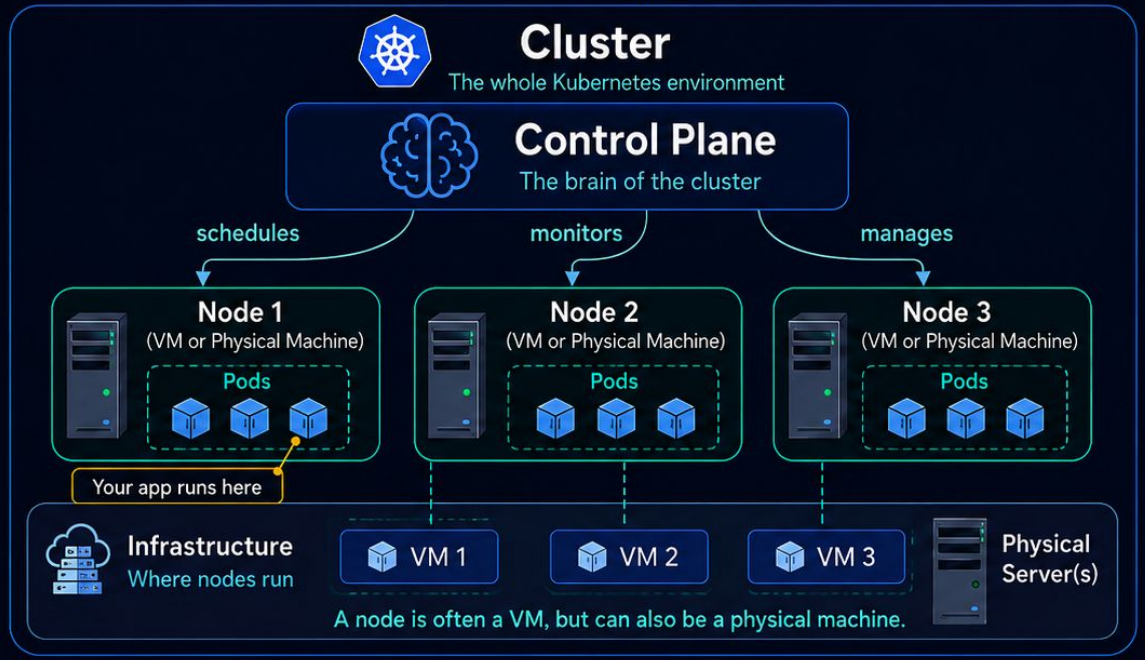
Term	Simple Meaning
Rolling Update	Gradually replaces old Pods with new ones
Rollback	Reverts to a previous application version
Readiness Probe	Checks whether a Pod is ready to receive traffic
Liveness Probe	Checks whether a Pod should be restarted
Startup Probe	Checks whether a slow-starting app has started successfully
Horizontal Pod Autoscaler	Automatically increases or decreases Pod replicas
Resource Request	Minimum CPU/memory a container asks for
Resource Limit	Maximum CPU/memory a container can use
Taint	Marks a node so normal Pods avoid it
Toleration	Allows a Pod to run on a tainted node

Operational Terms… …

Term	Simple Meaning
Affinity	Rules for where Pods should be placed
NodeSelector	Simple way to choose specific nodes
Pod Disruption Budget	Ensures enough Pods stay available during maintenance

Understanding Cluster, Node, and Control Plane

A simple way to think about Kubernetes



1. Cluster

Everything together: control plane + nodes

2. Control Plane

The manager/brain.
It decides where apps run and keeps the system healthy.

3. Node

A worker machine.
It runs your application Pods.
A node is often a VM, but it may also be a physical server.

Quick Note

VM = Virtual Machine

In many real clusters, each node is a VM.

Cluster
= the whole office

The entire office environment.

Control Plane
= the manager

The manager decides where work happens and keeps everything running.

Node
= a worker desk

The desk sits inside the office infrastructure (often a VM, but can be a physical machine) and runs your work (Pods).

Understanding Multi-Cluster

A cluster is one complete Kubernetes environment. Multi-cluster means several separate Kubernetes environments.

One Cluster

Cluster A

Control Plane
Manages the worker nodes

- Worker Node 1 — Pods
- Worker Node 2 — Pods
- Worker Node 3 — Pods

One cluster = one control plane + its worker nodes.

Multi-Cluster Example

- **Dev Cluster** — Control Plane (Manages nodes) — Pods Pods Pods
- **Staging Cluster** — Control Plane (Manages nodes) — Pods Pods Pods
- **Production Cluster** — Control Plane (Manages nodes) — Pods Pods Pods

Each cluster is independent.

Why use multiple clusters?

- Environment isolation (dev / staging / prod)
- Different regions or data centers
- Safer operations and fault isolation

Naive Analogy

One cluster
= one office with one manager and many worker desks.

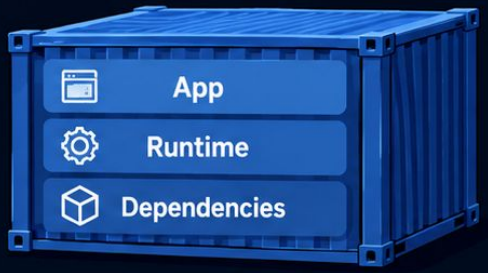
Multi-cluster
= several offices, each with its own manager and workers.

Cluster = one managed Kubernetes world. Multi-cluster = many separate Kubernetes worlds.

Understanding Container and Pod

A simple way to think about Kubernetes

Container



A container is one packaged application unit.

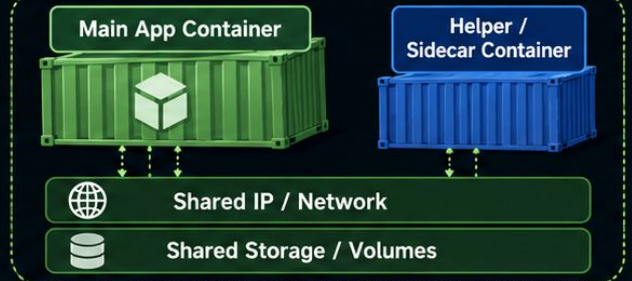
- Runs one application process
- Has its own filesystem and process space
- Can run by itself

Think: one packaged app

Kubernetes runs containers inside Pods.

Kubernetes does not manage loose containers directly. It manages Pods.

Pod



A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers.

- Usually contains one main container
- Can contain more than one tightly related container
- Containers in a Pod share network
- Containers in a Pod can share storage

Think: a wrapper that holds one or more containers

Container = Lunch Box Item

One packaged item.

Pod = Lunch Box

A holder that groups related items together.

Technical analogy: Container = app unit | Pod = deployment unit

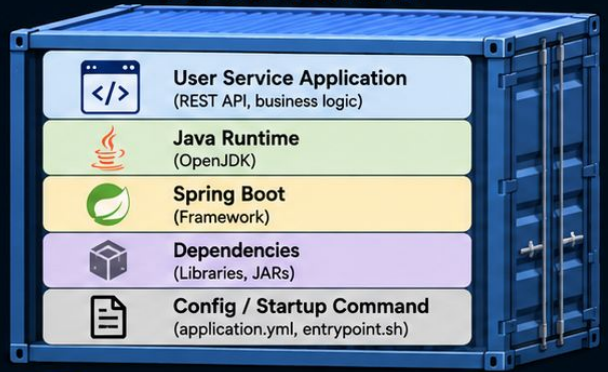
Container = the application package. Pod = the Kubernetes wrapper that runs one or more containers together.

Understanding Container and Pod with a Real Example

Example: User Service

Container

User Service Container



A container is one packaged application unit.
It usually runs one main process.

- Contains the User Service app
- Can run by itself
- Has its own process and filesystem

Think: one packed food item

Kubernetes runs containers inside Pods.

Pod

User Service Pod

User Service Container

- **REST API** (HTTP endpoints)
- **Business Logic** (Core features)
- **Database Calls** (Read / Write)

Log/Monitoring Sidecar
(Helper Container)

- Collects Logs
- Sends Metrics
- Exports Data

Shared IP / Network
(Same IP address and ports)

Shared Volume / Storage
(Files can be shared between containers)

A Pod is the smallest deployable unit in Kubernetes.
It wraps one or more containers that work together.

- Usually has one main app container
- May include a helper/sidecar container
- Containers inside share network
- Containers inside can share storage

Think: one lunch box holding related items together

Container = the packaged User Service app

Pod = the Kubernetes wrapper that runs the User Service and related helper container(s) together

Deployment & ReplicaSet

How Kubernetes makes sure the right number of Pod replicas are always running

Real-world Analogy

Deployment = Restaurant Manager

You tell the manager how many chefs you need.

ReplicaSet = Head Chef

The head chef ensures the exact number of chefs are working.

Pod (Replica) = Chef

Chefs (pods) actually cook and serve.

KUBERNETES EXAMPLE

Deployment (Desired State)

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
spec:
  replicas: 3   # I want 3
  template:
    ... (Pod definition)
```

Creates

ReplicaSet

(Maintains the Desired State)

RS

```
selector: app: user-service
replicas: 3
```

Ensures 3 Pod replicas are running at all times.

Manages

Pods (Replicas)

Pod	Pod	Pod
app: user-service version: v1	app: user-service version: v1	app: user-service version: v1
Running	Running	Running

If a pod fails, ReplicaSet creates a new one
If extra pods exist, ReplicaSet removes them

What Happens If...

- **A Pod fails?** ReplicaSet creates a new one automatically.
- **You scale to 5?** ReplicaSet creates 2 more pods.
- **You scale to 1?** ReplicaSet removes extra pods.

How It Works

You define a Deployment → Deployment creates a ReplicaSet → ReplicaSet creates and monitors Pods → Desired number of Pods is always maintained

Key Takeaway

Deployment defines WHAT you want (3 replicas of user-service)

ReplicaSet ensures THAT happens and stays that way.

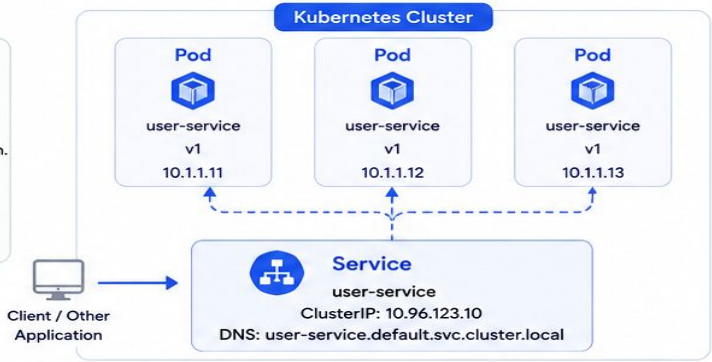
Service

Gives Pods a stable IP and DNS name so they can be reached reliably

Real-world Analogy

Imagine a Restaurant

Customers don't go to individual chefs in the kitchen. They talk to the restaurant (counter) which stays the same, even if chefs change.



What it does

- Provides a stable IP and DNS name for a set of pods
- Load balances traffic across healthy pods
- If a pod dies and a new one comes up, Service automatically sends traffic to the new pod

Key Takeaway

Service = Stable address + Load Balancing for a group of Pods

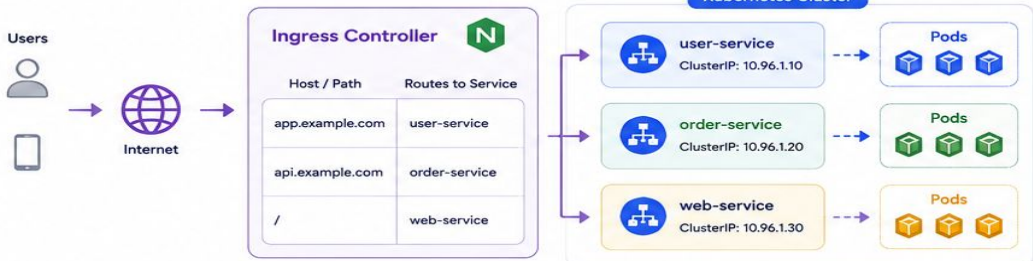
Ingress

Manages external HTTP/HTTPS traffic and routes it to the right Service

Real-world Analogy

Imagine a Mall

People enter the mall from the main gate. The security/desk (Ingress) checks the store name in the request and directs them to the right store.



What Ingress does

- Single entry point (one public IP / domain)
- Routes traffic based on host names and paths
- Works with TLS/SSL for secure access
- Hides internal services from the outside world

End-to-End Flow Example

ConfigMap

Stores non-sensitive configuration as key-value pairs.
Applications can use this configuration without rebuilding the image.

Real-world Analogy

Restaurant Menu Board

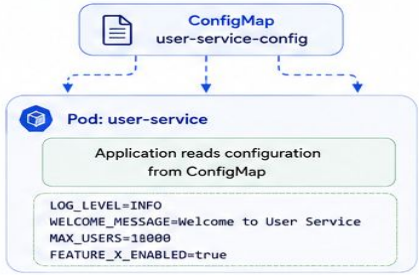
The menu (configuration) can change daily without changing the kitchen (application code).

Kubernetes Example

ConfigMap YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-service-config
data:
  log-level: "INFO"
  welcome-message: "Welcome to User Service"
  max-users: "1000"
  feature-x-enabled: "true"
```

How a Pod uses ConfigMap



Use Cases

- Application settings (log level, timeouts, URLs)
- Feature flags
- Environment-specific configurations
- Text files, templates, JSON, properties, etc.

Key Takeaway ConfigMap is for non-sensitive data. It helps you change configuration without rebuilding your application image.

Secret

Stores sensitive information such as passwords, tokens, API keys.
Kubernetes stores it in an encoded (Base64) format.

Real-world Analogy

Restaurant Safe

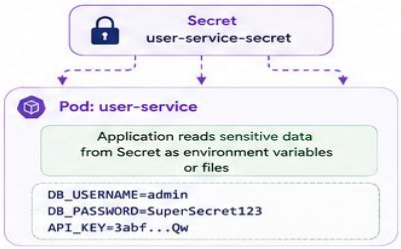
Sensitive information like the safe combination or bank credentials are kept in a safe, not on the menu board.

Kubernetes Example

Secret YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: user-service-secret
type: Opaque
data:
  db-username: YWRtaW4=          # base64 for "admin"
  db-password: U3VpZXJTZWCyZXQxMIIM==   # base64
  api-key: M2FiZ..Vw             # base64
```

How a Pod uses Secret



Use Cases

- Database credentials
- API keys and tokens
- OAuth / JWT secrets
- TLS certificates

Important Notes

- Secrets are Base64 encoded, not encrypted.
- Anyone with access to the cluster can decode them.
- Enable RBAC to restrict access.

Best Practice

Use Secrets for sensitive data and ConfigMaps for everything else.
Never hardcode secrets in your application or images.

At a Glance

	ConfigMap	Secret
Purpose	Non-sensitive configuration	Sensitive information
Stored as	Plain text (key-value)	Base64 encoded
Example	log-level, app URL, timeout	password, token, API key

Volume, PersistentVolume & PersistentVolumeClaim

Kubernetes storage building blocks that work together to store data beyond the life of a Pod.

1 VOLUME

A Volume is storage that is mounted into a Pod.
It is ephemeral – it exists as long as the Pod exists.

Real-world Analogy

A lunch box for a single meal.
When the meal (Pod) is over, the lunch box and food go away.

Example

emptyDir Volume
Temporary space inside the node.
Data is lost when Pod is deleted.

Pod
/app/data
(emptyDir)

2 PERSISTENTVOLUME (PV)

A PersistentVolume is a piece of storage in the cluster provisioned by an admin. It exists independently of any Pod.

Real-world Analogy

A hard drive on the server.
It exists on the server, regardless of who is using it.

Example

PersistentVolume (PV)
Capacity: 10Gi
Access Modes: ReadWriteOnce
Storage Class: standard
Reclaim Policy: Retain
Status: Available

10Gi

3 PERSISTENTVOLUMECLAIM (PVC)

A PersistentVolumeClaim is a request for storage by a user.
Kubernetes finds a matching PV and binds it to the PVC.

Real-world Analogy

A request slip to use a hard drive.
The slip (PVC) is matched to an available hard drive (PV).

Example

PersistentVolumeClaim (PVC)
Request: 10Gi
Access Modes: ReadWriteOnce
Storage Class: standard
Status: Bound

10Gi

How They Work Together

1 Admin creates PV
Admin provisions storage in the cluster.

PersistentVolume
10Gi
Status: Available

2 User creates PVC
User requests storage using a PVC.

PersistentVolumeClaim
Request: 10Gi
Status: Pending

3 PVC binds to PV
Kubernetes finds a matching PV and binds it to the PVC.

PVC
Status: Bound

PV
Status: Bound

4 Pod uses the PVC
Pod uses the PVC as a Volume.
Data is stored in the PV.

Pod
Volume Mount
/app/data
(from PVC)
PVC
PV

Real-life Example

Scenario
A web application needs to store user uploads.
The data must persist even if the Pod restarts.

Pod
/app/uploads

PVC
Request: 20Gi
Status: Bound

PV
20Gi

- If the Pod is deleted and recreated, it can use the same PVC.
- The PVC is already bound to the PV, so the data is safe.
- The data lives on the storage (PV), not inside the Pod.

Volume Types

Ephemeral (Pod lifetime)
- emptyDir
- configMap
- secret
- projected
- hostPath

Persistent (Beyond Pod lifetime)
- persistentVolumeClaim
- csi (cloud storage, NFS, etc.)

Key Takeaway

PV is the storage in the cluster.
PVC is the request for that storage.
Volume is how the Pod uses it.
Together, they provide persistent, reliable storage for stateful applications.

At a Glance

Component	Created By	Purpose	Lifetime
Volume	User (in Pod spec)	Mounted in a Pod	As long as Pod exists
PersistentVolume (PV)	Admin	Actual storage in the cluster	Independent of Pod
PersistentVolumeClaim (PVC)	User	Request storage and bind to PV	Independent of Pod

Kubernetes Probes

Probes help Kubernetes know the health of your containers and take action when needed.

1. Readiness Probe

Is the container ready to serve traffic?

Kubernetes uses this to decide whether to send traffic to the Pod.

Use case (Example: User Service)

User Service needs time to load data and warm up cache. Until it's ready, don't send user requests.

What happens?

- If the probe succeeds → Pod is Ready → receives traffic
- If the probe fails → Pod is Not Ready → traffic is not sent

Example (HTTP GET)

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
  successThreshold: 1
```

2. Liveness Probe

Is the container alive?

Kubernetes uses this to know when to restart the container.

Use case (Example: User Service)

If User Service is stuck (deadlock, infinite loop), Liveness probe fails and Kubernetes restarts it.

What happens?

- If the probe succeeds → Kubernetes leaves the container running
- If the probe fails → Kubernetes kills the container and restarts it

Example (HTTP GET)

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

3. Startup Probe

Has the application started?

Used for slow-starting applications.
Disables Liveness/Readiness checks until it succeeds.

Use case (Example: User Service)

User Service may take 60–90s to start (DB migrations, cache load). Don't kill it during this time.

What happens?

- Kubernetes runs Startup probe until it succeeds
- After success → Readiness & Liveness probes start
- If startup probe fails → Container is restarted

Example (HTTP GET)

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 30
```

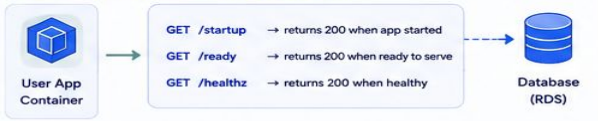
How They Work Together (User Service Lifecycle)

1. **Pod Created**
   Containers start.
2. **Startup Probe runs**
   Kubernetes checks /startup until it succeeds.
3. **Readiness & Liveness start**
   - Readiness → controls traffic
   - Liveness → ensures it stays alive
4. **Running Normally**
   Traffic is served. Probes keep checking.
5. **If Liveness Fails**
   Kubernetes restarts the container.

Probe Types

HTTP GET	Calls an HTTP endpoint (e.g., /healthz)
TCP Socket	Checks if a TCP port is open
Exec	Runs a command inside the container

Example: User Service (Spring Boot App)



Best Practices

- Use Startup probe for apps with long boot time.
- Set timeouts and failure thresholds carefully.
- Keep probes lightweight and fast.
- Use meaningful endpoints (not just /).
- Monitor probe failures in logs and metrics.

At a Glance

Probe	Purpose	When It Runs	What Happens on Failure
Readiness	Is the app ready to serve traffic?	After container starts	Pod marked NotReady, traffic not sent
Liveness	Is the app alive?	After container starts (after Startup succeeds)	Container is restarted
Startup	Has the app started?	Right after container starts	Container is restarted (if it keeps failing)

Kubernetes Requests, Limits & Autoscaling for a User Service

Right amount of resources for your containers. Scale automatically based on demand.

1. Requests

Reserve the resources your container needs.

- Scheduler uses requests to decide which node can run the Pod.
- Ensure the container gets at least the requested resources.

Example (User Service Container)

```
resources:
  requests:
    cpu:    "250m"      # 0.25 CPU core
    memory: "256Mi"     # 256 MiB RAM
```

2. Limits

Set the maximum resources the container can use.

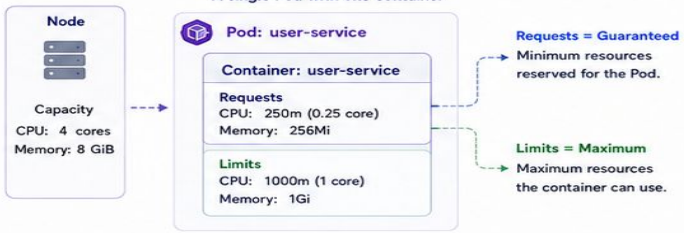
- Prevents a container from using more resources than allowed.
- If a container tries to exceed memory limit – it will be killed.

Example (User Service Container)

```
resources:
  limits:
    cpu:    "1000m"     # 1 CPU core
    memory: "1Gi"       # 1 GiB RAM
```

Example: User Service Pod

A single Pod with one container



Key Takeaway

Requests help with scheduling and guarantee minimum resources. Limits protect the cluster from resource overuse.

What Happens?

Situation	CPU	Memory
Uses less than request	Use what it needs	Use what it needs
Uses between request & limit	Use as needed (up to limit)	Use as needed (up to limit)
Exceeds limit	Throttled (CPU)	Container OOMKilled (killed)

Best Practices

- Set requests based on application needs.
- Set limits to prevent noisy neighbor problem.
- Keep requests <= limits.
- Review and tune based on monitoring.

3. Autoscaling

Automatically adjusts capacity based on demand.

- Helps handle increases in traffic.
- Saves cost by scaling down when demand is low.

Types of Autoscaling

- **HPA** – Horizontal Pod Autoscaler (scales Pods)
- **VPA** – Vertical Pod Autoscaler (adjusts CPU/Memory requests and limits)
- **CA** – Cluster Autoscaler (scales Nodes)

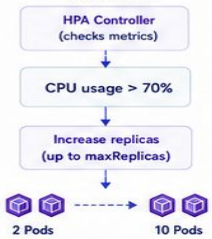
Horizontal Pod Autoscaler (HPA) — Scales Pod Count

Example: Scale User Service based on CPU usage

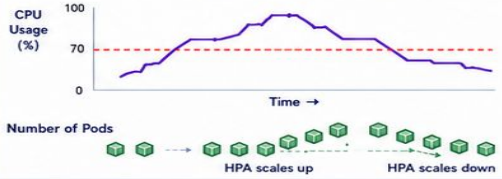
HPA Configuration

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: user-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: user-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # target 70% CPU
```

How it works



Visual Example (User Traffic Increases)



Metrics Sources

- CPU Utilization
- Memory Utilization
- Custom Metrics (e.g., requests/sec)
- External Metrics (e.g., queue length)

Vertical Pod Autoscaler (VPA)

Adjusts CPU/Memory requests and limits

- Watches resource usage
- Recommends or updates requests/limits
- Does not change Pod count

Cluster Autoscaler (CA)

Scales Nodes based on pending Pods

- If Pods are pending due to no resources, CA adds more nodes to the cluster
- Scales down nodes when not needed

End-to-End Flow (User Service)

Users send traffic → Load increases → CPU usage crosses 70% → HPA scales up Pods → More Pods handle the traffic → Stable performance maintained

Summary

- Use **Requests & Limits** to allocate and control resources.
- Use **Autoscaling (HPA/VPA/CA)** to adapt to changing demand.
- Monitor continuously and tune for optimal performance & cost.

Kubernetes: Namespace, Labels & Selectors

Organize resources. Add meaningful tags. Find the right resources. Once.

1. NAMESPACE

Namespaces provide a way to divide cluster resources between multiple users, teams, or environments.

Real-life Analogy

Think of a namespace as a department in a company. Each department has its own resources and team, separate from others, but all within the same company.

Example

A company uses namespaces to separate environments.


```
# Create a namespace
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace prod
```

If you don't specify a namespace, Kubernetes uses the default namespace.

2. LABELS

Labels are key-value pairs attached to Kubernetes objects. They do not provide uniqueness.

Real-life Analogy

Think of labels as tags on a product in an online store (e.g., color: blue, size: M). A product can have multiple tags, and tags can be reused.

Example

We add labels to our resources to describe them.

User Service Pod with Labels

```
apiVersion: v1
kind: Pod
metadata:
  name: user-service
  namespace: prod
  labels:
    app: user-service
    tier: backend
    environment: production
spec:
  containers:
  - name: user-service
    image: myapp/user-service:1.0
```

Labels on this Pod

Key	Value
app	user-service
tier	backend
environment	production

Labels help organize and identify resources. They are used by selectors to group objects.

3. SELECTORS

Selectors are used to find and group resources based on their labels.

Real-life Analogy

Think of selectors as search filters on an e-commerce website (e.g., show me all blue, size M shirts). It finds items (resources) matching the tags (labels).

Example

A Deployment uses a selector to find which Pods it should manage.

Deployment with Selector

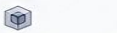
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  namespace: prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
        tier: backend
  spec:
    containers:
      - name: user-service
        image: myapp/user-service:1.0
```

This selector matches Pods that have the label:

`app: user-service`

Matching Pods

Non-matching Pods



Only Pods with matching labels are managed by this Deployment.

How They Work Together

1. **Create Namespaces** — Separate environments or teams.
2. **Add Labels to Resources** — Tag your resources with meaningful labels.
3. **Use Selectors to Group** — Controllers (Deployments, Services, etc.) use selectors to find matching resources.
4. **Kubernetes Manages Matching Pods** — Only the Pods with matching labels are managed.

Real-life Scenario: E-commerce App

Key Takeaways

- Namespace isolates and organizes resources.
- Labels add metadata (key-value) to resources.
- Selectors find and group resources based on labels.
- Labels do not provide uniqueness; selectors do the filtering.
- Together, they help build scalable, maintainable, and multi-tenant Kubernetes applications.

At a Glance

Concept	Purpose
Namespace	Organize resources into isolated groups
Labels	Attach key-value metadata to resources
Selectors	Find and group resources based on labels

1. Cloud Managed Kubernetes

This is the easiest production-grade option.

The cloud provider manages the Kubernetes control plane, upgrades, ensure high availability, and cloud integrations.

You mostly manage worker nodes, workloads, networking rules, CI/CD, security, monitoring, and cost.

1. Cloud Managed Kubernetes… …

Provider	Kubernetes Service	Best For
AWS	Amazon EKS	Enterprise, AWS-native systems, banking/fintech, mature ecosystem
Azure	AKS	Microsoft/.NET-heavy companies, Azure AD integration, enterprise apps
Google Cloud	GKE	Strong Kubernetes-native experience, autoscaling, platform engineering
DigitalOcean	DOKS	Simpler, cheaper, developer-friendly Kubernetes
Oracle Cloud	OKE	Oracle-heavy environments, Oracle DB ecosystem
IBM Cloud	IBM Cloud Kubernetes Service	Enterprise/hybrid use cases
Red Hat	OpenShift managed/cloud variants	Enterprise platform with Kubernetes plus developer/security tooling

2. Self-Managed Kubernetes on Cloud VMs

This means you rent cloud VMs but install Kubernetes yourself.

Example:

- AWS EC2 VMs + kubeadm/K3s/RKE2
- Azure VMs + kubeadm/K3s/RKE2
- Google Compute Engine + kubeadm/K3s/RKE2
- DigitalOcean Droplets + kubeadm/K3s

2. Self-Managed Kubernetes on Cloud VMs… …

Architecture example:

Cloud Provider

- ├── VM 1: Control Plane
- ├── VM 2: Worker Node
- ├── VM 3: Worker Node
- ├── Load Balancer
- ├── Block Storage
- └── Firewall / Security Group

2. Self-Managed Kubernetes on Cloud VMs… …

Benefit: You get more control and may reduce managed service cost.

Pain: You are now responsible for:

1. Control plane setup	2. etcd backup
3. Cluster upgrades	4. Certificate rotation
5. Networking plugin	6. Ingress controller
7. Storage class	8. Node failure recovery
9. Security hardening	10. Monitoring
11. Logging	12. Disaster recovery
Many more … …	Many more … …

3. Kubernetes Without Cloud

Option A — Local Kubernetes for Learning

Best for students, labs, and hands-on demos.

Tool	Best For
Minikube	Beginner learning, single-machine Kubernetes
kind	Fast local clusters, CI testing, Kubernetes experiments
Docker Desktop Kubernetes	Simple local development
MicroK8s	Local dev, edge, small production-like setup
K3s	Lightweight Kubernetes, labs, edge, small servers
K3d	K3s inside Docker, very fast local clusters

3. Kubernetes Without Cloud … …

Option B — On-Prem Kubernetes

This means Kubernetes runs inside your own company datacenter or server room.

Example:

Company Datacenter

- ├── Physical Server 1
- ├── Physical Server 2
- ├── Physical Server 3
- ├── Internal Load Balancer
- ├── Storage System
- ├── Firewall
- └── Monitoring Stack

3. Kubernetes Without Cloud … …

Option B — On-Prem Kubernetes

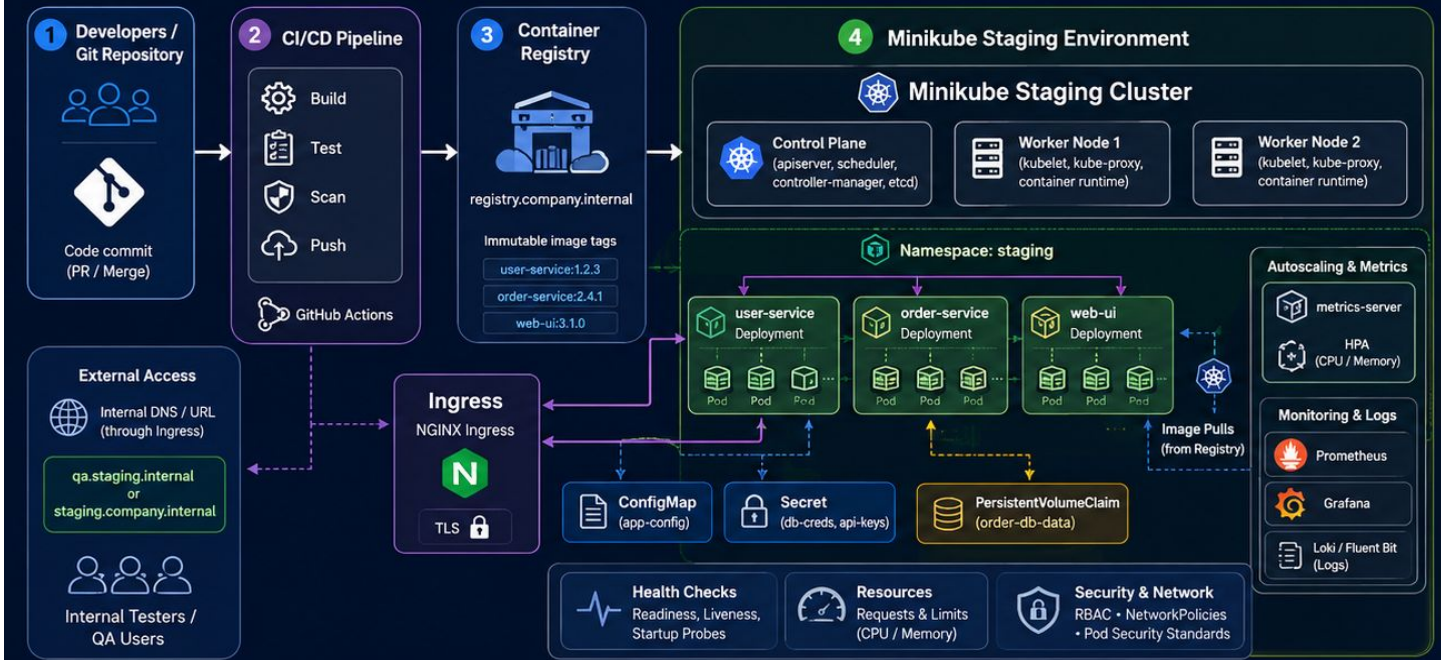
Common options:

Option	Description
kubeadm	Official-style manual Kubernetes setup
K3s	Lightweight production-capable Kubernetes
RKE2	Rancher Kubernetes Engine 2, enterprise/security-focused
MicroK8s	Simple Kubernetes from Canonical
OpenShift Container Platform	Enterprise Kubernetes platform
VMware Tanzu	Kubernetes for VMware-heavy enterprises
Rancher	Multi-cluster Kubernetes management platform

Using Minikube for a Staging Environment

Practical setup + best practices for small internal staging, demos, and pre-production testing

Best-practice note: Minikube is acceptable for lightweight internal staging, but real enterprise staging usually uses a multi-node or managed Kubernetes cluster.



Industry Best Practices

- Use a dedicated Minikube profile or dedicated host for staging
- Prefer namespace isolation: "staging"
- Deploy through CI/CD, not manual changes
- Use immutable image tags from a registry
- Use Ingress + TLS for internal access
- Store config in ConfigMaps and secrets securely
- Set CPU/memory requests and limits
- Enable readiness, liveness, and startup probes
- Enable metrics-server and HPA
- Use PVCs for stateful test data
- Apply RBAC and NetworkPolicies
- Monitor logs, metrics, and events

When Minikube is a Good Fit

- **Team demos** — Spin up quickly for demos and walkthroughs.
- **QA / UAT for small teams** — Lightweight staging for feature validation.
- **CI validation of Kubernetes manifests** — Test manifests and deployments end-to-end.
- **Cost-conscious internal staging** — Low-cost option for internal pre-prod needs.

Limitations

- **Not ideal for HA testing** — Single-node by default.
- **Limited production parity** — Missing multi-node, cloud integrations.
- **Single-host failure risk** — No multi-node redundancy.
- **Better to move to managed Kubernetes for serious staging** — Scales, secures, and operates better.

Thank you